Design Patterns: UML Answer Notes

Design pattern scenarios | CO2 | 4 marks per part

Question 1(a) – Writing Modes

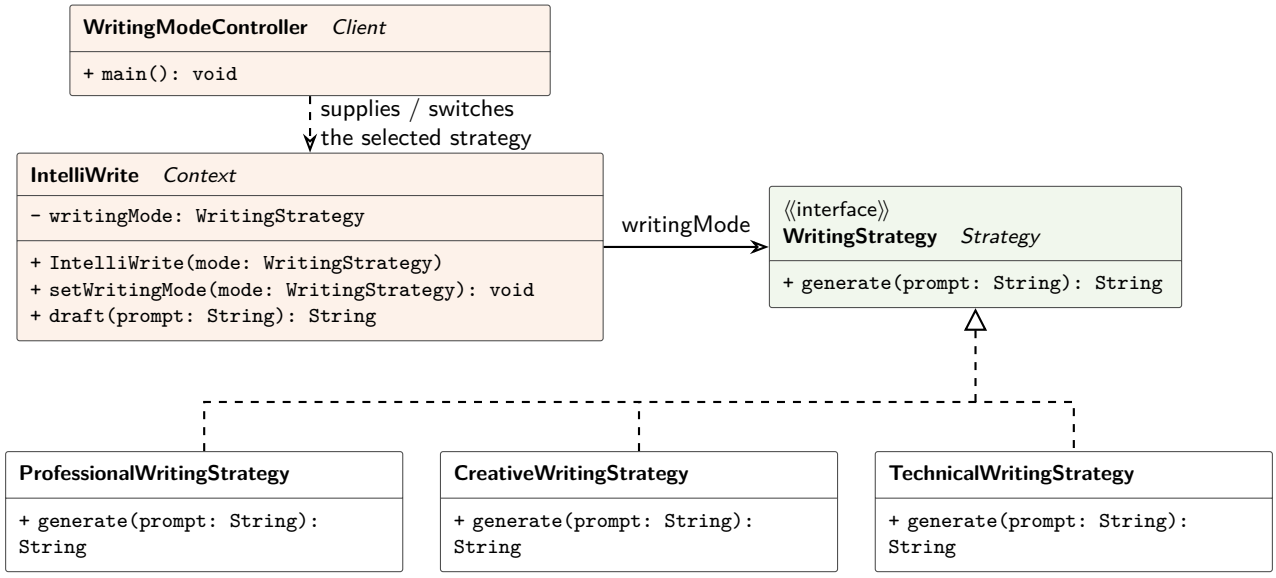
IntelliWrite is an AI-powered writing assistant used for drafting emails, reports, and social media posts. The application supports multiple text generation modes, including Professional Writing, Creative Writing, and Technical Writing. Users can switch between these generation modes at runtime depending on their needs. The system should allow additional writing modes to be added in the future without modifying existing code.

Task: Design a UML diagram using the appropriate design pattern.

Answer – Strategy Pattern (Behavioral)

Encapsulate each writing mode as a separate strategy implementing the same `WritingStrategy` interface. `IntelliWrite` holds the selected strategy and delegates text generation to it. The user can replace the strategy at runtime without changing the application's generation logic.

Detailed UML Class Diagram



UML notation: + public; − private. Solid arrow: navigable association. Dashed line with a hollow triangle: interface realization. Dashed arrow: dependency.

Reading the diagram: Each concrete mode realizes `WritingStrategy`. Every `IntelliWrite` instance has exactly one active strategy. A regular association is used because exclusive lifecycle ownership is not required by the scenario.

Delegation rule: `draft(prompt)` returns `writingMode.generate(prompt)`. Each concrete strategy implements its own writing behavior through `generate(prompt)`; no additional attributes are needed in this diagram.

Question 1(b) – Integrating a New AI Provider

IntelliWrite interacts with language models via the standard `LLMService` interface for text generation. The system was originally designed to work with OpenAI's GPT service. The team now wants to integrate Anthropic's Claude as an additional AI provider. Since Claude uses a different request and response format, the team must ensure that the new providers' format works seamlessly with the existing application without modifying IntelliWrite's internal code.

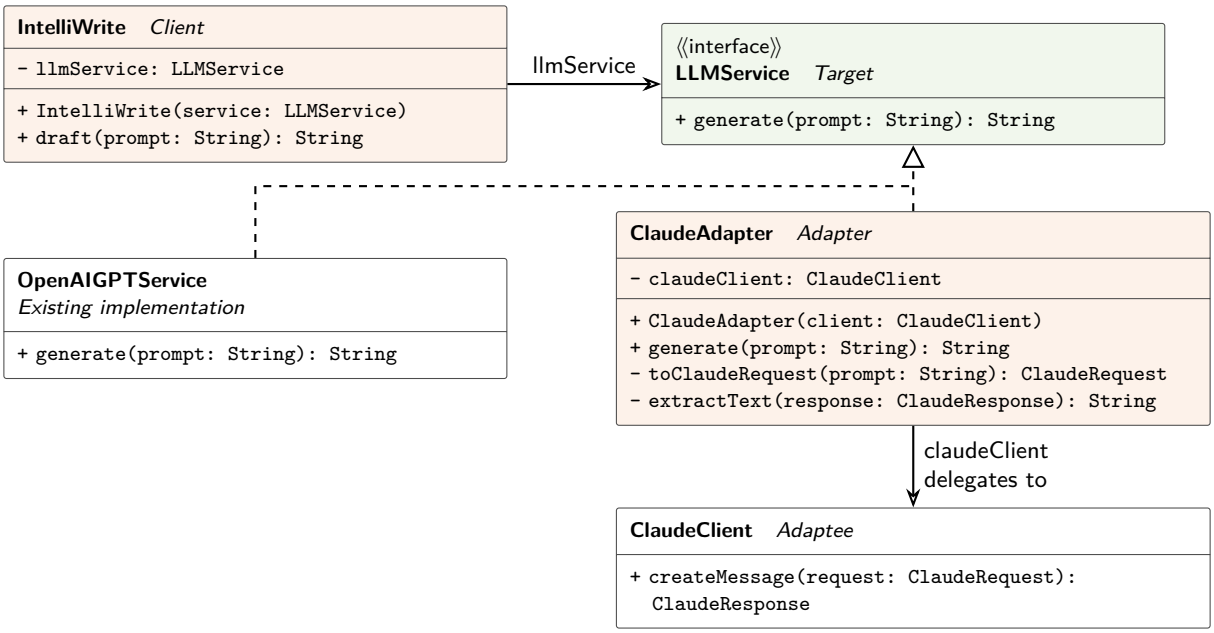
Task: Design a UML diagram using the appropriate design pattern.

Answer – Adapter Pattern (Structural)

Create a `ClaudeAdapter` that implements the existing `LLMService` interface and wraps a `ClaudeClient`. It converts the application's input into the provider-specific request, invokes Claude, and converts the response back into the application's expected result. `IntelliWrite` continues to use the same interface.

Detailed UML Class Diagram

Modeling assumption: The following signatures and data types are illustrative application-level abstractions, not actual OpenAI or Anthropic SDK methods. The existing GPT integration already implements `LLMService`; provider selection is configured outside IntelliWrite.



UML notation: + public; − private. Solid arrow: navigable association. Dashed line with a hollow triangle: interface realization. Dashed arrow: dependency.

Reading the diagram: Both provider implementations realize `LLMService`, but `ClaudeClient` does not. The adapter holds one Claude client and translates calls rather than changing the client or the application. The existing application receives the selected service through its existing dependency-injection mechanism.

Question 2(a) – FitRide Workout Calculations

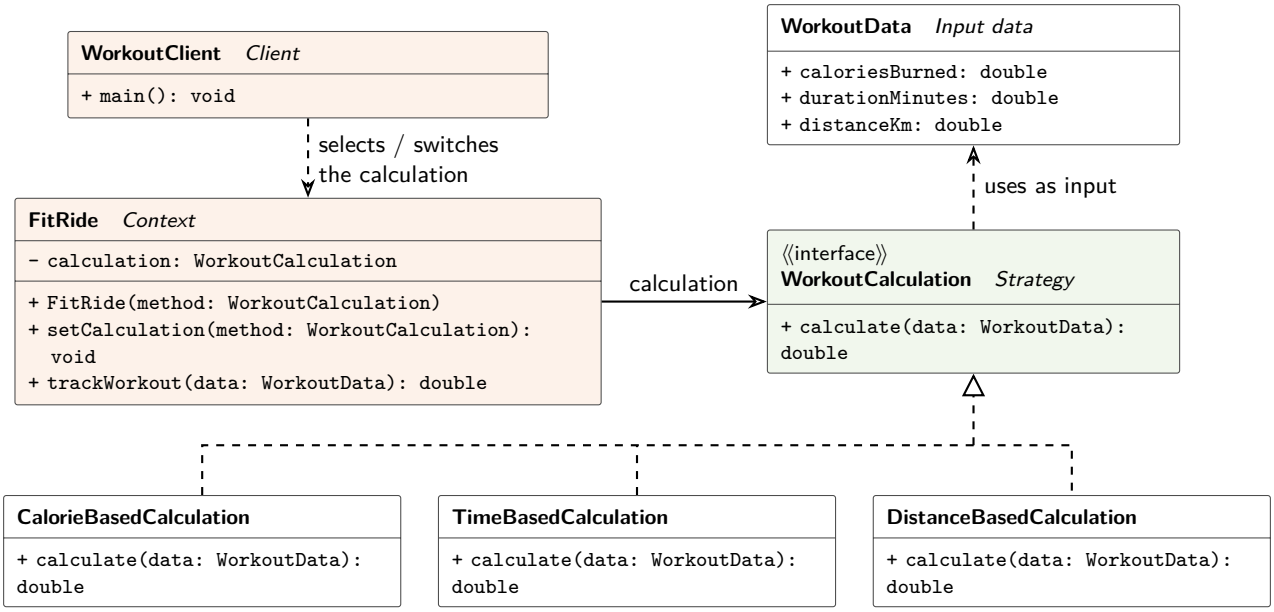
FitRide is a fitness application that allows users to track their workouts. The app supports various workout calculation techniques, including calorie-based, time-based, and distance-based workouts. A user can switch between these workout calculation techniques at runtime depending on their fitness goal. The application should allow adding new workout calculation techniques in the future without modifying existing code.

Task: Design a UML diagram using the appropriate design pattern. **[CO2, 4 marks]**

Answer – Strategy Pattern (Behavioral)

Define a common `WorkoutCalculation` interface and implement each calculation technique in a separate class. `FitRide` stores the selected technique and delegates calculation to it. The client can replace the technique at runtime without changing FitRide's calculation logic.

Detailed UML Class Diagram



UML notation: + public; – private. Solid arrow: navigable association. Dashed line with a hollow triangle: interface realization. Dashed arrow: dependency.

Reading the diagram: The three calculation classes implement `WorkoutCalculation`. `WorkoutData` contains the workout's recorded measurements. Each technique calculates the tracked result for its own metric: calories, minutes, or kilometres. The result's unit follows the selected technique.

Delegation rule: `trackWorkout(data)` returns `calculation.calculate(data)`. To switch techniques, the client calls `setCalculation` with another strategy object.

Question 2(b) – UniPay Payment Manager

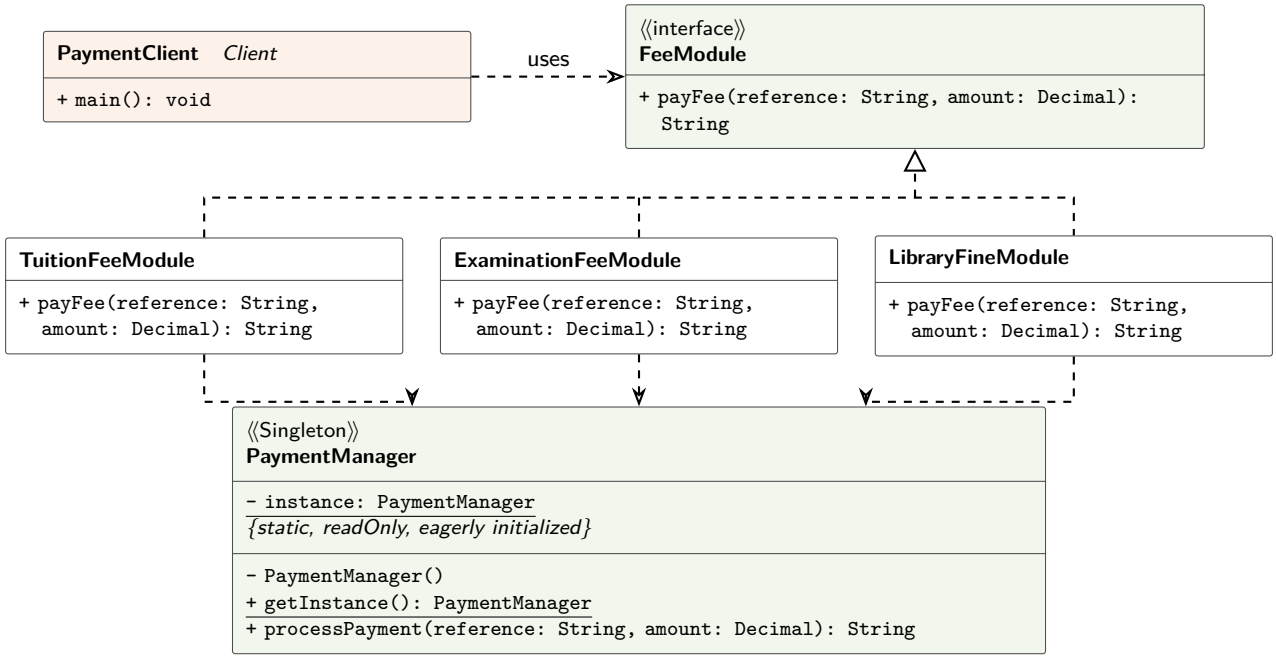
UniPay is a university-wide online payment system used for tuition fees, examination fees, and library fines. To avoid data inconsistency and transaction conflicts, the system must ensure that only one instance of the payment manager exists throughout the application lifecycle. All payment requests from different modules must be handled through this single instance.

Task: Design a UML diagram using the appropriate design pattern. **[CO2, 4 marks]**

Answer – Singleton Pattern (Creational)

Make `PaymentManager` a Singleton: keep its constructor private, store the shared instance in a private static field, and expose it through a public static `getInstance()` method. Every payment module obtains the same manager and sends its payment requests to that object. A separate `PaymentClient` accesses all fee modules through the common `FeeModule` interface.

Detailed UML Class Diagram



UML notation: + public; − private. Solid arrow: navigable association. Dashed line with a hollow triangle: interface realization. Dashed arrow: dependency.

Additional notation: Underlined members are static (class-level). `readOnly` means the stored instance reference is not reassigned after initialization.

Reading the diagram: `PaymentClient.main()` selects a fee module and calls its common `payFee` operation. Tuition, examination, and library modules implement `FeeModule`; all use the same `PaymentManager`. Its private constructor prevents direct creation, and static `getInstance()` always returns the shared instance.

Delegation rule: Each module's `payFee` method delegates to:

```
PaymentManager.getInstance().processPayment(reference, amount)
```

The reference identifies the fee or fine; the returned string is a transaction identifier. `Decimal` represents an exact decimal amount.